SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of conduct :
I R.Nivetha /1912260554 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R.Nivetha

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



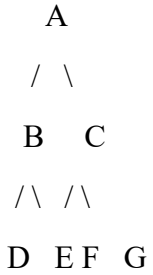
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Question 1: Breadth-First Search (BFS)

Graph: A is connected to B and C. B is connected to D and E. C is connected to F and G. Start node is A and neighbours are taken left to right. BFS uses a queue (FIFO): remove the front node, visit it, then add its unvisited children at the back. The queue column below shows the queue after each iteration is finished.

Iteration	Queue	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	Empty	G

1. BFS traversal order: A → B → C → D → E → F → G.

2. The queue contents after every iteration are in the table above. The queue starts as [A] and ends empty after G is visited. BFS goes level by level: level 0 is A, level 1 is B, C and level 2 is D, E, F, G.

Question 2: Depth-First Search(DFS)

Same graph, starting from A. DFS uses a stack (LIFO): pop the top node, visit it, then push its children. I push the right child first and the left child last, so the left child ends up on top and is explored first. The top of the stack is the right-most item.

Iteration	Stack (top at right)	Visited Node
1	C, B	A
2	C, E, D	B
3	C, E	D
4	C	E
5	G, F	C
6	G	F

Iteration	Stack (top at right)	Visited Node
7	Empty	G

DFS traversal order: A → B → D → E → C → F → G.
DFS finishes the whole left branch (B, D, E) before it ever touches C. The stack never held more than 3 nodes here, while the BFS queue held up to 4.

Question 3: BFS on the 8-Puzzle

Initial state: 1 3 6 / 5 0 2 / 4 7 8. **Goal state:** 1 2 3 / 4 5 6 / 7 8 0. 0 is the blank tile.

The blank can move Up, Down, Left or Right, and every move costs 1. BFS keeps a FIFO queue and a visited list, so a state that was already generated is not added again (for example, when S1 is expanded, moving the blank Down just gives back S0, so it is skipped). States are named S0, S1, S2... in the order they are generated. Their layouts are in the key below.

State key (generated from)

S0	S1	S2	S3	S4	S5	S6
1 3 6 5 0 2 4 7 8	1 0 6 5 3 2 4 7 8	1 3 6 5 7 2 4 0 8	1 3 6 0 5 2 4 7 8	1 3 6 5 2 0 4 7 8	0 1 6 5 3 2 4 7 8	1 6 0 5 3 2 4 7 8
S7	**S8**	**S9**	**S10**	**S11**	**S12**	
1 3 6 5 7 2 0 4 8	1 3 6 5 7 2 4 8 0	0 3 6 1 5 2 4 7 8	1 3 6 4 5 2 0 7 8	1 3 0 5 2 6 4 7 8	1 3 6 5 2 8 4 7 0	

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	1 3 6 5 0 2 4 7 8	S0	S1, S2, S3, S4

Iteration	Current State	Queue Before Expansion	Queue After Expansion
2	1 0 6 5 3 2 4 7 8	S1, S2, S3, S4	S2, S3, S4, S5, S6
3	1 3 6 5 7 2 4 0 8	S2, S3, S4, S5, S6	S3, S4, S5, S6, S7, S8
4	1 3 6 0 5 2 4 7 8	S3, S4, S5, S6, S7, S8	S4, S5, S6, S7, S8, S9, S10
5	1 3 6 5 2 0 4 7 8	S4, S5, S6, S7, S8, S9, S10	S5, S6, S7, S8, S9, S10, S11, S12

The first five iterations are shown above as asked. The search carries on in the same way, one level at a time, until the goal comes to the front of the queue.

Questions

1. Nodes expanded: S0, S1, S2, S3 and S4 in iterations 1 to 5, in the order they were added to the queue. Overall, the goal sits at depth 8 and it is the 311th state taken off the queue, so 310 states are expanded before the goal is reached.

2. States generated: the first 5 iterations generate 4 + 2 + 2 + 2 + 2 = 12 new states (13 with S0). For the full search up to the goal, 499 new states are generated (500 counting the initial state), after skipping repeated states.

3. Solution path (blank moves): Right, Up, Left, Down, Left, Down, Right, Right. That is 8 moves, and the states are:

Start	Right	Up	Left	Down	Left	Down	Right	Right

1 3 6 5 0 2 4 7 8	1 3 6 5 2 0 4 7 8	1 3 0 5 2 6 4 7 8	1 0 3 5 2 6 4 7 8	1 2 3 5 0 6 4 7 8	1 2 3 0 5 6 4 7 8	1 2 3 4 5 6 0 7 8	1 2 3 4 5 6 7 0 8	1 2 3 4 5 6 7 8 0

4. Why BFS gives the shortest solution: BFS expands every state at depth d before any state at depth d+1. In the 8-puzzle each move costs the same (1), so the depth of a state is exactly its path cost. The first time the goal shows up, it must be at the smallest possible depth, because every shallower state was already checked and was not the goal. The state space is finite and the branching factor is at most 4, so BFS is also complete and will always find a solution if one exists.